FIBER CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & APP

Installation

```
go get github.com/gofiber/fiber/v2
app := fiber.New(fiber.Config{
Prefork: true,           // multi-process
BodyLimit: 4 * 1024 * 1024, // 4MB
})
app.Listen(":3000")
app.ListenTLS(":443", "cert.pem", "key.pem")
```

02. ROUTING

Architecture

```
app.Get("/users", listUsers)
app.Post("/users", createUser)
app.Put("/users/:id", updateUser)
app.Delete("/users/:id", deleteUser)
app.All("/debug", handler)  // all methods
app.Use(handler)  // matches all paths
```

03. CTX METHODS

YAML / Config

```
c.Params("id")           // route param
c.Query("page", "1")    // query string with ...
c.Body()                 // raw request body
c.BodyParser(&req)       // unmarshal body to...
return c.JSON(data)      // send JSON
return c.Status(201).JSON(data)
return c.SendString("text")
return c.SendFile("./file.pdf")
```

04. MIDDLEWARE

Workflow

```
import 'github.com/gofiber/fiber/v2/middlewar...
app.Use(logger.New())
app.Use(recover.New())
app.Use(cors.New(cors.Config{AllowOrigins: "*...
// Custom:
app.Use(func(c *fiber.Ctx) error {
c.Locals("requestID", uuid.New())
return c.Next()
})
```

05. GROUPING

CRUD API

```
api := app.Group("/api")
v1 := api.Group("/v1", JWTMiddleware())
v1.Get("/users", listUsers)
admin := v1.Group("/admin", AdminOnly())
admin.Get("/stats", getStats)
```

Group middleware applies to all routes in group

WORKSHEET DOWNLOAD

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. STATIC FILES

Hardening

```
app.Static("/", "./public")
app.Static("/public", "./public", fiber.Stati...
Compress: true,
CacheDuration: 10 * time.Second,
MaxAge: 3600,
})
// SPA fallback:
app.Static("*", "./public/index.html")
```

08. BODY PARSING

Observability

```
type User struct {
Name string `json:"name" validate:"required"`
}
var u User
if err := c.BodyParser(&u); err != nil {
return c.Status(400).JSON(fiber.Map{"error": ...
}
// File upload:
file, _ := c.FormFile("document")
c.SaveFile(file, "uploads/"+file.Filename)
```

09. ERROR HANDLING

Optimization

```
app.Use(func(c *fiber.Ctx) error {
err := c.Next()
if err != nil {
code := fiber.StatusInternalServerError
if e, ok := err.(*fiber.Error); ok { code = e...
return c.Status(code).JSON(fiber.Map{"error":...
}
return nil
})
return fiber.NewError(fiber.StatusBadRequest,...
```

10. COMMON CTX METHODS

Unit Tests

```
c.Locals("key", val)   // request-scoped stor...
c.Locals("key")        // retrieve
c.IP()                  // client IP
c.Get("Authorization") // request header
c.Set("X-Custom", val) // response header
c.Method()             // GET POST etc
c.Path()               // URL path
```

11. COMMON GOTCHAS

Commands

Fiber uses fasthttp net/http handlers incompatible
c.Body() returns []byte, not io.Reader save before parsing
Prefork mode: goroutines/channels don't work across processes
c.Locals() cleared after request don't use as global cache
JSON tags required on structs unexported fields ignored

WORKSHEETDOWNLOAD